TP 02 — Simuler la propagation d'une épidémie

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

De Vinci Higher Education — ESILV

ESILV
DE VINCI ENGINEERING SCHOOL

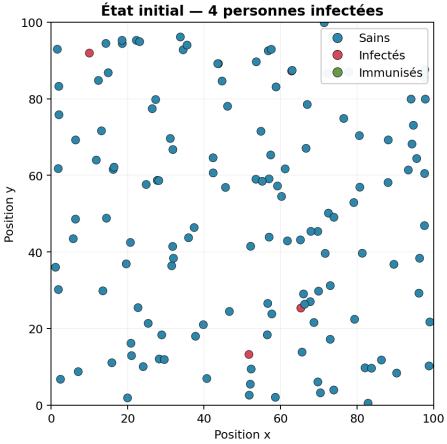
Notre mission

Question

Comment des règles individuelles très simples peuvent-elles produire une épidémie à l'échelle d'une population ?

Chaque point représente une personne qui possède :

- une position (x, y) ;
- une vitesse (v_x, v_y) ;
- un état sanitaire.



D'un point de vue algorithmique

Une génération de l'épidémie enchaîne quatre opérations :

```
def creer_population():
    # Initialiser positions, vitesses et etats.
def deplacer_individus(population):
    # Avancer et rebondir sur les bords.
def contaminer_population(population):
    # Tester les contacts avec les infectes.
def mettre_a_jour_infections(population):
    # Faire passer les gueris a l'etat immunise.
```

La visualisation répète ensuite ces opérations à chaque image.

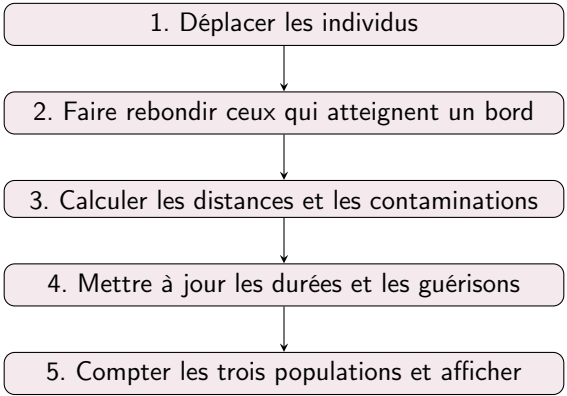
Trois états possibles



Simplification du modèle

Une personne immunisée ne redevient pas malade. Le modèle décrit une idée algorithmique ; il ne cherche pas à prévoir une véritable épidémie.

Une génération de la simulation



Correspondance entre les étapes et les TODO

Avant de lancer la boucle, les **TODO 1 à 3** créent la population initiale : positions, vitesses et premiers individus infectés.

1. **Déplacer les individus**	**TODO 4**
2. **Faire rebondir sur les bords**	**TODO 5 et 6**
3. **Calculer les distances et les contaminations**	**TODO 7 à 10**
4. **Mettre à jour les durées et les guérisons**	**TODO 11 et 12**
5. **Compter les trois populations**	**TODO 13**

Affichage

Le rafraîchissement des graphiques est déjà fourni : il utilise les positions, les états et les compteurs calculés pendant ces cinq étapes.

Créer la population — TODO 1 à 3

Les tableaux NumPy contiennent une ligne par personne et deux colonnes pour les coordonnées x et y.

```
# TODO 1 : positions dans le carre [0, TAILLE_MONDE]
positions = np.random.uniform(
    0, TAILLE_MONDE, size=(NB_INDIVIDUS, 2)
)

# TODO 2 : composantes de vitesse
vitesses = np.random.uniform(
    -VITESSE_MAX, VITESSE_MAX,
    size=(NB_INDIVIDUS, 2)
)

# TODO 3 : indices distincts, sans remise
infectes = np.random.choice(
    NB_INDIVIDUS, NB_INFECTES_INITIAUX, replace=False
)
```

Déplacer les individus — TODO 4

Pour chaque personne i, le nouveau point est obtenu par une addition de vecteurs :

$$x_i^{n+1} = x_i^n + v_i.$$

```
# positions et vitesses ont la meme forme : (NB_INDIVIDUS, 2)
positions += vitesses
```

Intérêt de NumPy

Une seule instruction réalise ici les 140 additions vectorielles.

Rebondir sur les bords — TODO 5 et 6

Un masque booléen sélectionne toutes les personnes ayant franchi une limite.

```
for axe in (0, 1):
    # TODO 5 : rebond sur le bord inferieur
    masque_bas = positions[:, axe] < 0
    positions[masque_bas, axe] = 0
    vitesses[masque_bas, axe] *= -1

    # TODO 6 : rebond sur le bord superieur
    masque_haut = positions[:, axe] > TAILLE_MONDE
    positions[masque_haut, axe] = TAILLE_MONDE
    vitesses[masque_haut, axe] *= -1
```

Inverser une composante de vitesse transforme v en $-v$: la personne repart vers l'intérieur du monde.

Mesurer la proximité — TODO 7 à 9

Pour une personne saine en position x_s et un infecté en position x_i :

$$\Delta x = x_i - x_s, \qquad d = \|\Delta x\|_2 = \sqrt{(x_i - x_s)^2 + (y_i - y_s)^2}.$$

1. **TODO 7** : extraire les indices tels que `etat == INFECTE` ;
2. **TODO 8** : calculer tous les vecteurs Δx ;
3. **TODO 9** : calculer leurs normes avec `axis=1`.

Mesurer la proximité — implémentation

```python
indices_infectes = np.where(etats == INFECTE)[0]

for i in np.where(etats == SAIN)[0]:
    differences = (
        positions[indices_infectes] - positions[i]
    )
    distances = np.linalg.norm(differences, axis=1)
```

Si cinq personnes sont infectées, `differences` a la forme $(5, 2)$ et `distances` contient cinq nombres.

Décider une contamination — TODO 10

Deux conditions sont nécessaires : un contact proche et un tirage aléatoire favorable.

```
if (
    np.any(distances < RAYON_CONTAMINATION)
    and np.random.random() < PROBABILITE_CONTAMINATION
):
    nouveaux_infectes.append(i)
```

Pourquoi attendre la fin de la boucle ?

Les nouveaux états sont appliqués ensemble, afin d'éviter une réaction en chaîne artificielle au cours d'une seule génération.

Guérir et devenir immunisé — TODO 11 et 12

```python
# TODO 11 : incrementer les durees d'infection
durees[etats == INFECTE] += 1

# TODO 12 : identifier les personnes gueries
gueris = (
    (etats == INFECTE)
    & (durees >= DUREE_INFECTION)
)
etats[gueris] = IMMUNISE
```

L'opérateur & combine deux tableaux de booléens élément par élément.

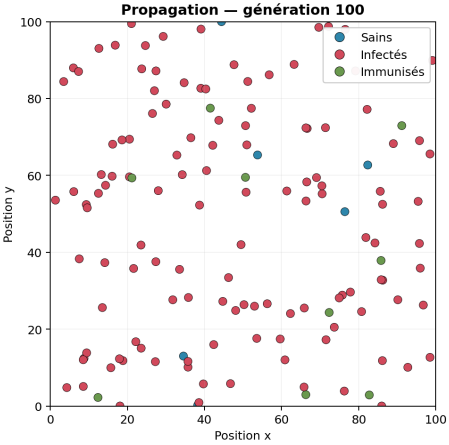
Compter la population — TODO 13

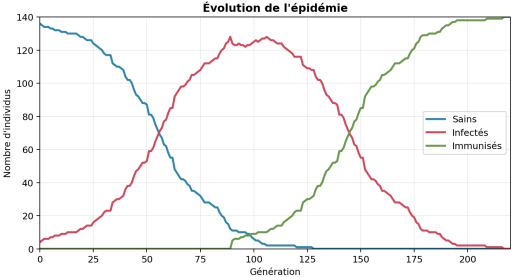
```python
return (
    int(np.sum(etats == SAIN)),
    int(np.sum(etats == INFECTE)),
    int(np.sum(etats == IMMUNISE)),
)
```

Vérification toujours vraie :

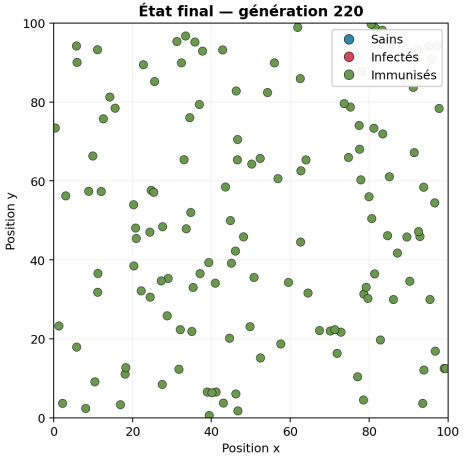
$$N_{\text{sains}} + N_{\text{infectes}} + N_{\text{immunises}} = N_{\text{individus}}.$$

Observer la propagation





État final



À retenir

Des règles locales simples font apparaître une dynamique globale : croissance, pic, puis disparition de l'infection.

Le résultat varie d'une exécution à l'autre car les déplacements et les contaminations sont aléatoires.

Expérimenter

Relancer la simulation après avoir modifié un seul paramètre :

- doubler `RAYON_CONTAMINATION` ;
- diviser `PROBABILITE_CONTAMINATION` par deux ;
- raccourcir `DUREE_INFECTION` ;
- ralentir les déplacements avec `VITESSE_MAX`.

Question

Quel paramètre modifie le plus la hauteur et la date du pic d'infection ?

Lancer le TP dans Spyder

1. Ouvrir `02-propagation_epidemie.py`.
2. Choisir un backend graphique **Automatic** ou **Qt**.
3. Redémarrer la console IPython si le backend vient d'être modifié.
4. Compléter les TODO dans l'ordre et exécuter le fichier après chaque groupe d'étapes.

Affichage dynamique

L'instruction `plt.show(block=True)` maintient la fenêtre ouverte pendant l'animation.

Concepts appris

- tableaux NumPy ;
- positions et vecteurs ;
- masques booléens ;
- distance euclidienne ;
- conditions combinées ;
- probabilités ;
- évolution d'un état ;
- visualisation dynamique.

À vous de coder !